Lecture 39

# Compression

**CS61B, Spring 2026 @ UC Berkeley**

Josh Hug and Kay Ousterhout

# Today's Goal: Compression

Lecture 39, CS61B, Spring 2026

## Today's Goal: Compression

Prefix Free Codes

Shannon Fano Codes (Extra)

Huffman Coding

- Core Idea
- Data Structures for Huffman Coding
- Huffman Coding in Practice

Compression Theory

- Compression is Hard
- Extreme Compression

LZW Style Compression (Extra)

# Zip Files, How Do They Work?

```
$ zip mobydick.zip mobydick.txt
  adding: mobydick.txt (deflated 59%)
$ ls -l
-rw-rw-r-- 1 jug jug 643207 Apr 24 10:55 mobydick.txt
-rw-rw-r-- 1 jug jug 261375 Apr 24 10:55 mobydick.zip
```

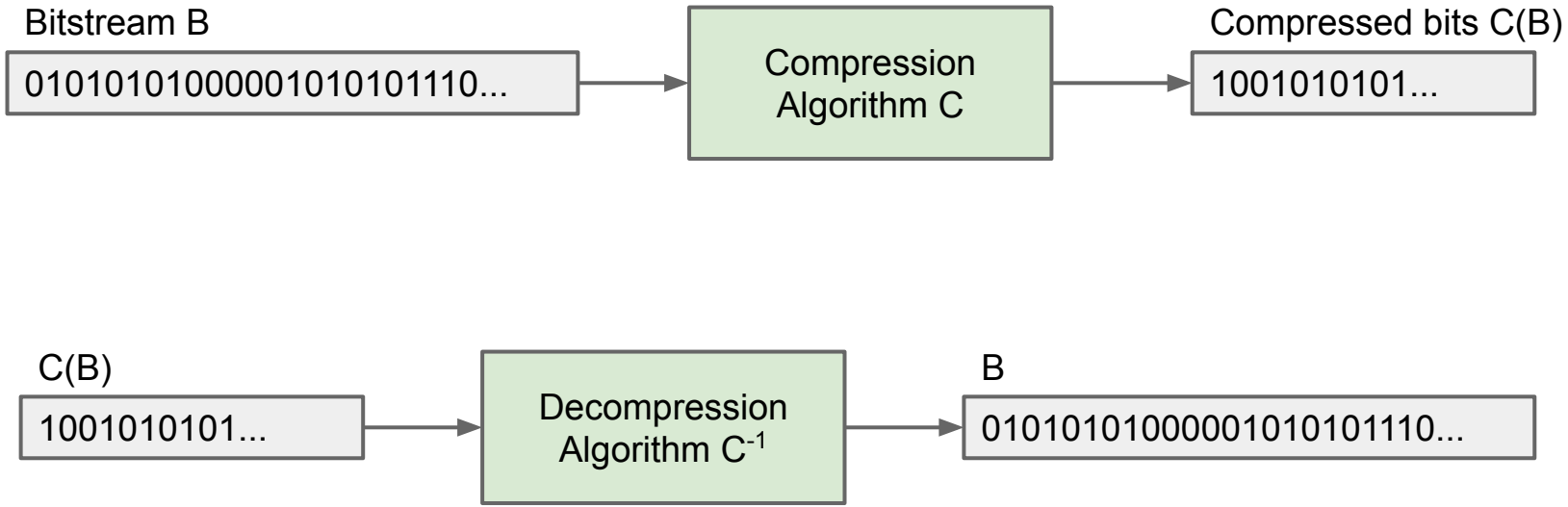
Size in Bytes

File is unchanged by zipping / unzipping.



```
$ unzip mobydick.zip
replace mobydick.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: r
new name: unzipped.txt
  inflating: unzipped.txt
$ diff mobydick.txt unzipped.txt
$
```

# Compression Model #1: Algorithms Operating on Bits




In a ***lossless*** algorithm we require that no information is lost.

- Text files often losslessly compressible by 70% or more.

# Prefix Free Codes

Lecture 39, CS61B, Spring 2026

Today's Goal: Compression

**Prefix Free Codes**

Shannon Fano Codes (Extra)

Huffman Coding

- Core Idea
- Data Structures for Huffman Coding
- Huffman Coding in Practice

Compression Theory

- Compression is Hard
- Extreme Compression

LZW Style Compression (Extra)

# Increasing Optimality of Coding

One common encoding for English text is to represent each letter by a sequence of 8 bits, e.g. ‘d’ is 01100100.

| word | binary | hexadecimal |
|---|---|---|
| dog | 01100100 01101111 01100111 | 64 6f 67 |

Easy way to compress: Use fewer than 8 bits for each letter.

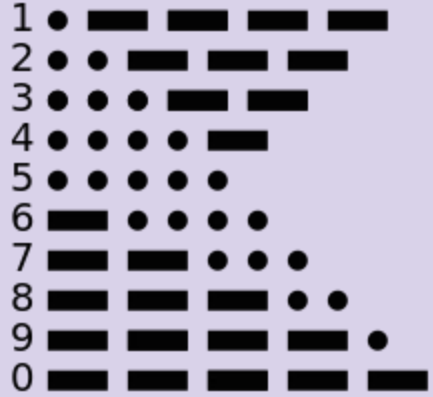
- Have to decide which bit sequences should go with which letters.
- More generally, we’d say which ***codewords*** go with which ***symbols.***

# More Code: Mapping Alphanumeric Symbols to Codewords: yellkey.com/human

Example: Morse code.

- Goal: Compact representation.
- What is – – • – – •?





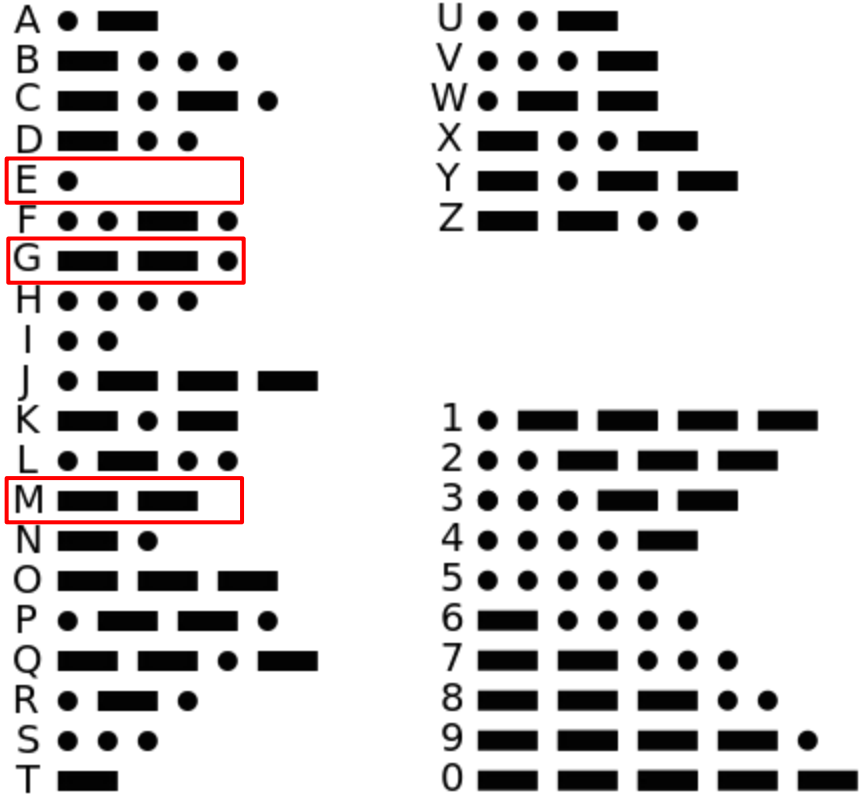
# More Code: Mapping Alphanumeric Symbols to Codewords

Example: Morse code.

- Goal: Compact representation.
- What is – – • – – •? It's ambiguous!
  - MEME
  - GG

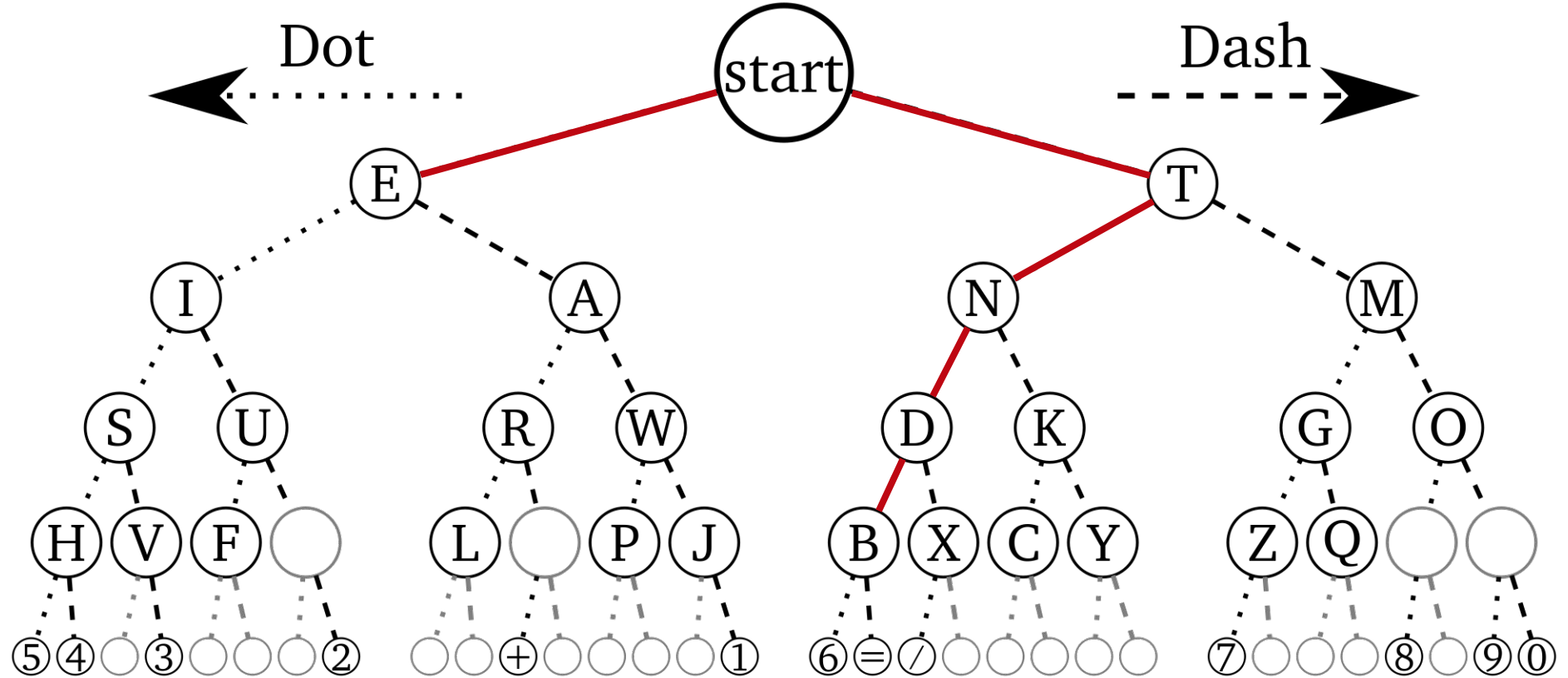
Note:

- Can think of dot as 0, dash as 1.
- Operators pause between codewords to avoid ambiguity.
  - Pause acts as a 3rd symbol.




Alternate strategy: Avoid ambiguity by making code ***prefix free***.

# Morse Code (as a Tree)

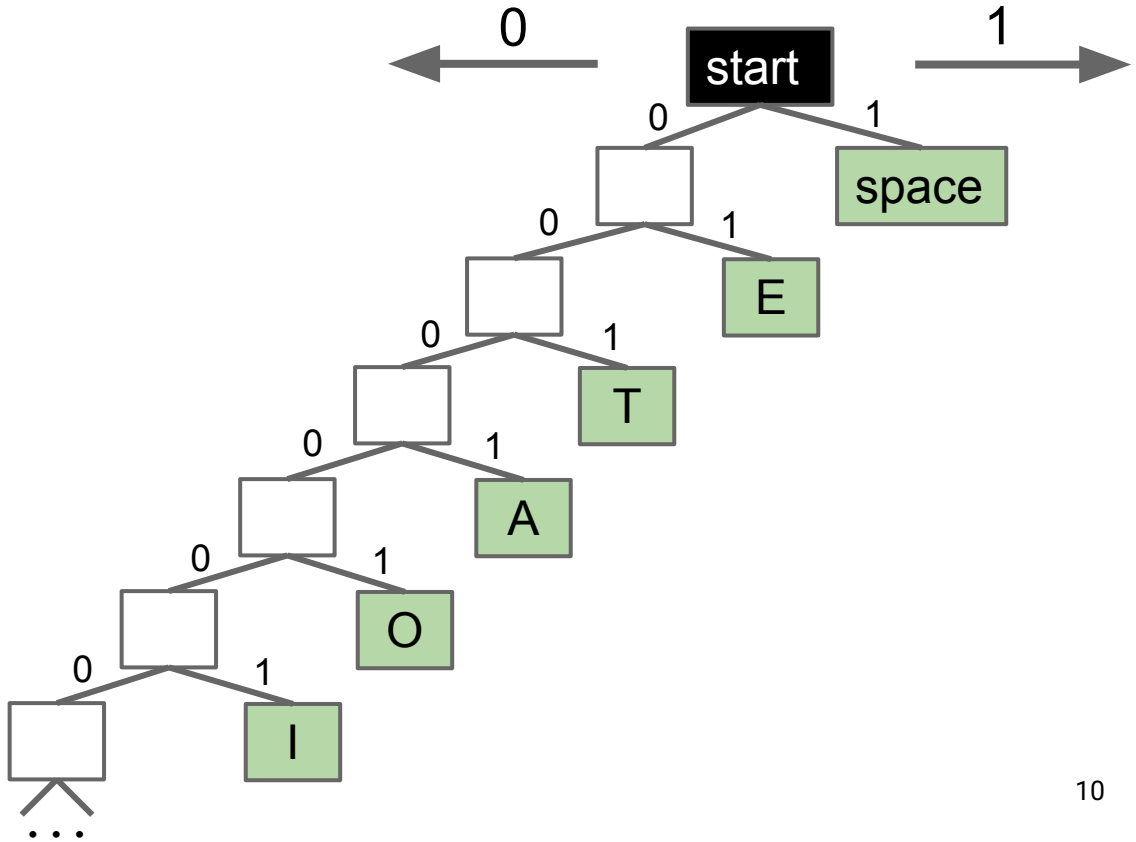



From Wikimedia

# Prefix-Free Codes [Example 1]

A prefix-free code is one in which no codeword is a prefix of any other. Example for English:

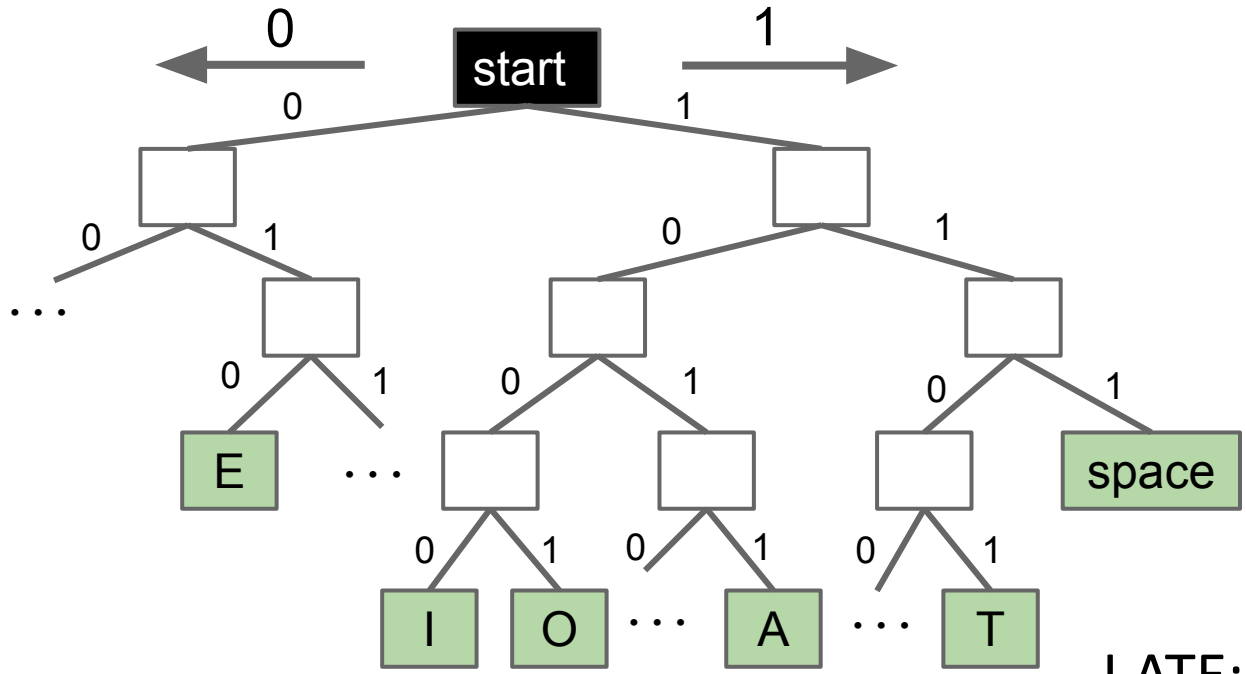
| | |
|---|---|
| space | 1 |
| E | 01 |
| T | 001 |
| A | 0001 |
| O | 00001 |
| I | 000001 |
| ... | |




I ATE: 000001100010010 1

# Prefix-Free Codes [Example 2]

A prefix-free code is one in which no codeword is a prefix of any other. Example for English:




| | |
|---|---|
| space | 111 |
| E | 010 |
| T | 1101 |
| A | 1011 |
| O | 1001 |
| I | 1000 |
| ... | |

I ATE: 10001111011110101 0

# Prefix Free Code Design

**Observation:** Some prefix-free codes are better for some texts than others.

Better for EEEEAT
(8+4+3 = 15 bits).

Much worse for JOSH
(25+5+8+10 = 48 bits).

| | |
|---|---|
| space | 1 |
| E | 01 |
| T | 001 |
| A | 0001 |
| O | 00001 |
| I | 000001 |
| ... | |

| | |
|---|---|
| space | 111 |
| E | 010 |
| T | 1101 |
| A | 1011 |
| O | 1001 |
| I | 1000 |
| ... | |

Worse for EEEEAT
(12+4+4 = 20 bits).

Better for JOSH
(7+4+6+6 = 23 bits).

**Observation:** It'd be useful to have a procedure that calculates the "best" code for a given text.

# Shannon Fano Codes (Extra)

Lecture 39, CS61B, Spring 2026

Today's Goal: Compression

Prefix Free Codes

**Shannon Fano Codes (Extra)**

Huffman Coding

- Core Idea
- Data Structures for Huffman Coding
- Huffman Coding in Practice

Compression Theory

- Compression is Hard
- Extreme Compression

LZW Style Compression (Extra)

# Code Calculation Approach #1 (Shannon-Fano Coding)

- Count relative frequencies of all characters in a text.
- Split into ‘left’ and ‘right halves’ of roughly equal frequency.
  - Left half gets a leading zero. Right half gets a leading one.
  - Repeat.

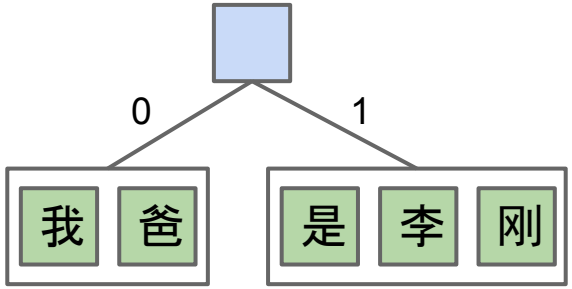
| | Symbol | Frequency |
|---|---|---|
| Left half | 我 | 0.35 |
| | 爸 | 0.17 |
| Right half | 是 | 0.17 |
| | 李 | 0.16 |
| | 刚 | 0.15 |

35% of all characters are 我

# Code Calculation Approach #1 (Shannon-Fano Coding)

- Count relative frequencies of all characters in a text.
- Split into ‘left’ and ‘right halves’ of roughly equal frequency.
  - Left half gets a leading zero. Right half gets a leading one.
  - Repeat.

| | Symbol | Frequency | Code |
|---|---|---|---|
| Left half | 我 | 0.35 | 0... |
| | 爸 | 0.17 | 0... |
| Right half | 是 | 0.17 | 1... |
| | 李 | 0.16 | 1... |
| | 刚 | 0.15 | 1... |



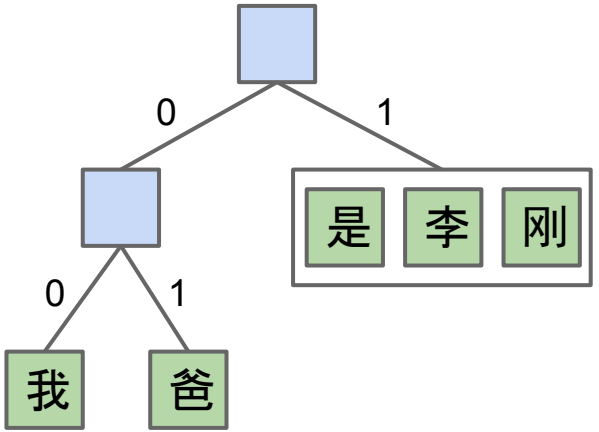
# Code Calculation Approach #1 (Shannon-Fano Coding)

- Count relative frequencies of all characters in a text.
- Split into ‘left’ and ‘right halves’ of roughly equal frequency.
  - Left half gets a leading zero. Right half gets a leading one.
  - Repeat.

Left half

Right half

| Symbol | Frequency | Code |
|---|---|---|
| 我 | 0.35 | 00 |
| 爸 | 0.17 | 01 |
| 是 | 0.17 | 1... |
| 李 | 0.16 | 1... |
| 刚 | 0.15 | 1... |



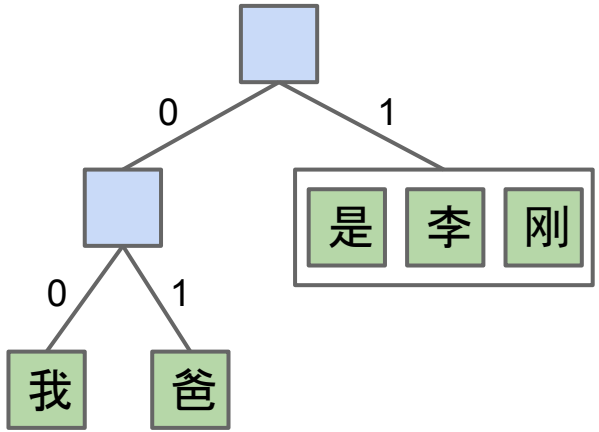
# Code Calculation Approach #1 (Shannon-Fano Coding)

- Count relative frequencies of all characters in a text.
- Split into ‘left’ and ‘right halves’ of roughly equal frequency.
  - Left half gets a leading zero. Right half gets a leading one.
  - Repeat.

| Symbol | Frequency | Code |
|---|---|---|
| 我 | 0.35 | 00 |
| 爸 | 0.17 | 01 |
| 是 | 0.17 | 1... |
| 李 | 0.16 | 1... |
| 刚 | 0.15 | 1... |

Left half

Right half



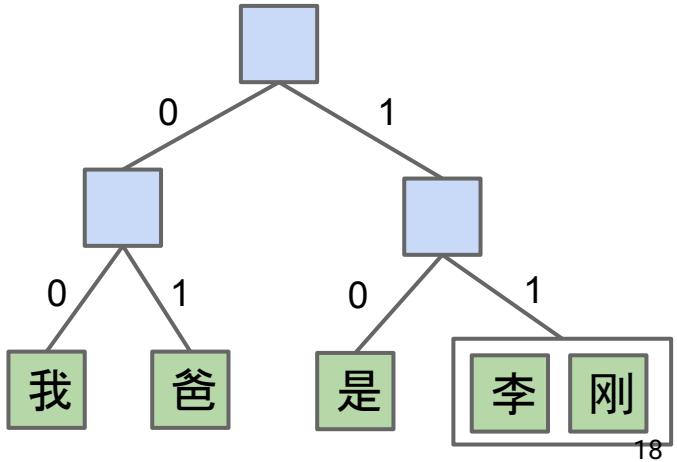
# Code Calculation Approach #1 (Shannon-Fano Coding)

- Count relative frequencies of all characters in a text.
- Split into ‘left’ and ‘right halves’ of roughly equal frequency.
  - Left half gets a leading zero. Right half gets a leading one.
  - Repeat.

| Symbol | Frequency | Code |
|---|---|---|
| 我 | 0.35 | 00 |
| 爸 | 0.17 | 01 |
| 是 | 0.17 | 10 |
| 李 | 0.16 | 11... |
| 刚 | 0.15 | 11... |

Left half

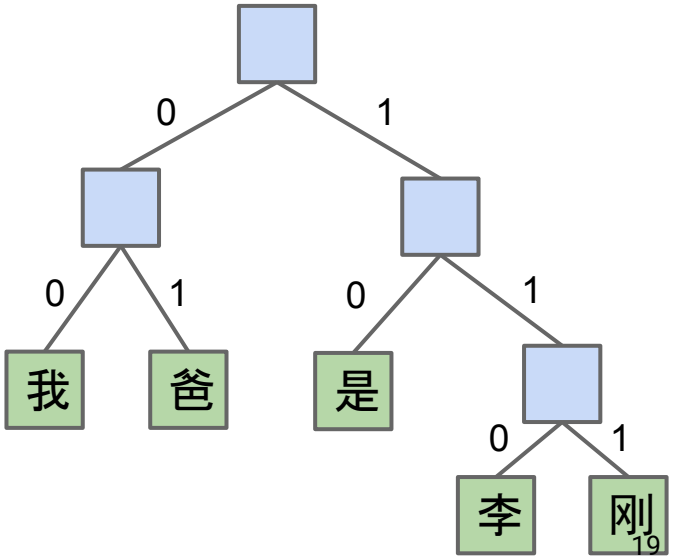
Right half



# Code Calculation Approach #1 (Shannon-Fano Coding)

- Count relative frequencies of all characters in a text.
- Split into ‘left’ and ‘right halves’ of roughly equal frequency.
  - Left half gets a leading zero. Right half gets a leading one.
  - Repeat.

| Symbol | Frequency | Code |
|---|---|---|
| 我 | 0.35 | 00 |
| 爸 | 0.17 | 01 |
| 是 | 0.17 | 10 |
| 李 | 0.16 | 110 |
| 刚 | 0.15 | 111 |

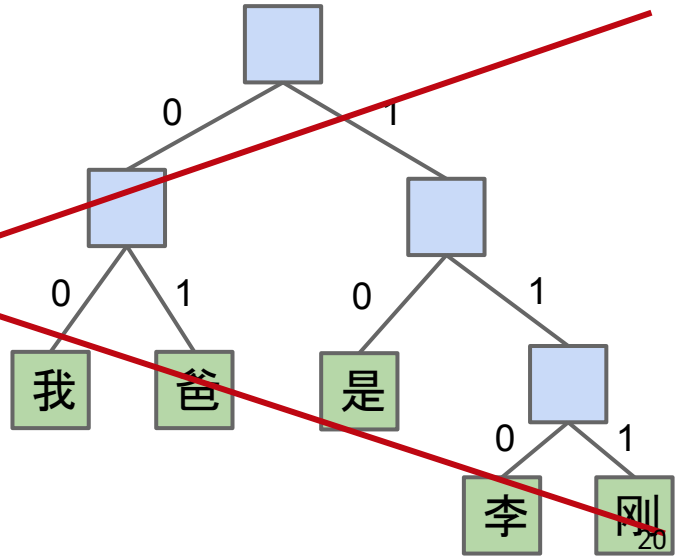


# Code Calculation Approach #1 (Shannon-Fano Coding)

Shannon-Fano coding is NOT optimal. Does a good job, but possible to find 'better' codes (see CS170).

- Optimal solution assigned (and solved) as alternative to a final exam: http://www.huffmancoding.com/my-uncle/scientific-american

| Symbol | Frequency | Code |
|---|---|---|
| 我 | 0.35 | 00 |
| 爸 | 0.17 | 01 |
| 是 | 0.17 | 10 |
| 李 | 0.16 | 110 |
| 刚 | 0.15 | 111 |



# Huffman Coding: Core Idea

Lecture 39, CS61B, Spring 2026

Today's Goal: Compression

Prefix Free Codes

Shannon Fano Codes (Extra)

**Huffman Coding**

- **Core Idea**
- Data Structures for Huffman Coding
- Huffman Coding in Practice

Compression Theory

- Compression is Hard
- Extreme Compression

LZW Style Compression (Extra)

# Code Calculation Approach #2: Huffman Coding

Suppose I have a text file with the 5 chinese characters shown.

- 35% of the characters are 我.

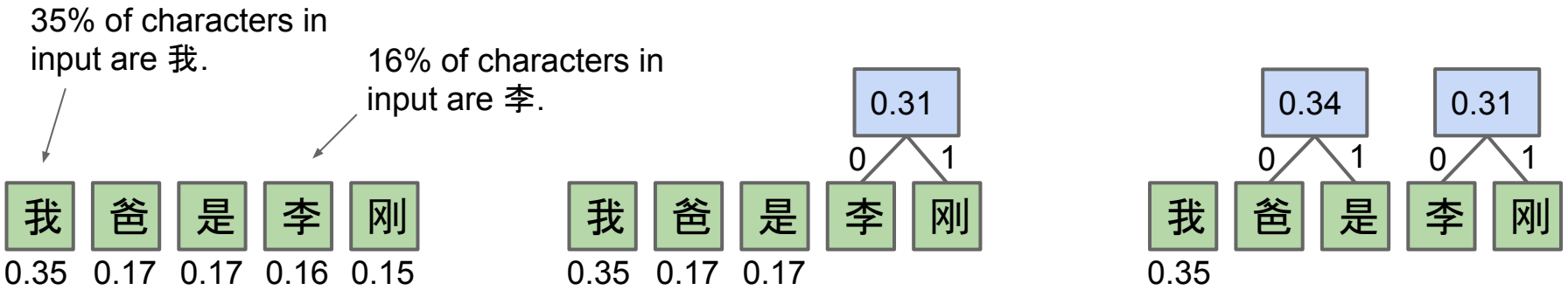
| Symbol | Frequency |
|---|---|
| 我 | 0.35 |
| 爸 | 0.17 |
| 是 | 0.17 |
| 李 | 0.16 |
| 刚 | 0.15 |

35% of all characters are 我

# Code Calculation Approach #2: Huffman Coding

Calculate relative frequencies.

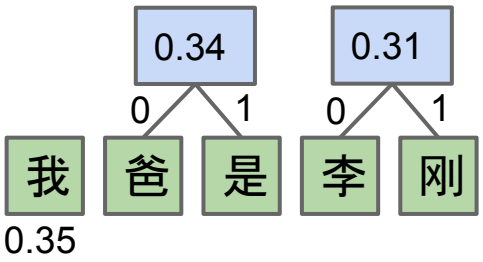
- Assign each symbol to a node with weight = relative frequency.
- Take the two smallest nodes and merge them into a super node with weight equal to sum of weights.
- Repeat until everything is part of a tree.

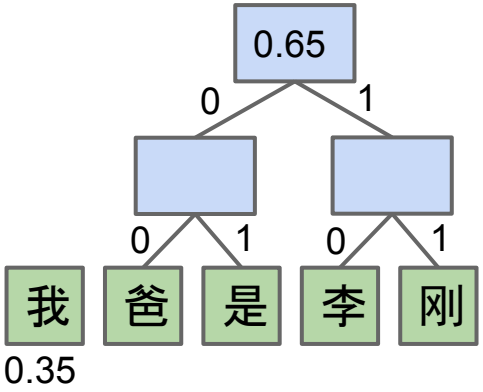


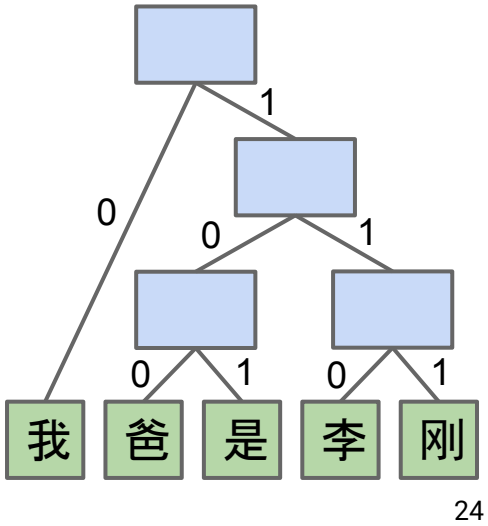
# Code Calculation Approach #2: Huffman Coding

Calculate relative frequencies.

- Assign each symbol to a node with weight = relative frequency.
- Take the two smallest nodes and merge them into a super node with weight equal to sum of weights.
- Repeat until everything is part of a tree.



# Efficiency Assessment: http://yellkey.com/east

How many bits per symbol do we need to compress a file with the character frequencies listed below using the Huffman code that we created?

A. (1*1 + 4*3) / 5
= 2.6 bits per symbol

B. (0.35) * 1 + (0.17 + 0.17 + 0.16 + 0.15) * 3
= 2.3 bits per symbol

C. Not enough information, we need to know the exact characters in the file being compressed.

| Symbol | Frequency | Huffman Code |
|---|---|---|
| 我 | 0.35 | 0 |
| 爸 | 0.17 | 100 |
| 是 | 0.17 | 101 |
| 李 | 0.16 | 110 |
| 刚 | 0.15 | 111 |

# Efficiency Assessment of Huffman Coding

How many bits per symbol do we need to compress a file with the character frequencies listed below using the Huffman code that we created?

**B. (0.35) * 1 + (0.17 + 0.17 + 0.16 + 0.15) * 3  = 2.3 bits per symbol.**

| Symbol | Frequency | Huffman Code |
|---|---|---|
| 我 | 0.35 | 0 |
| 爸 | 0.17 | 100 |
| 是 | 0.17 | 101 |
| 李 | 0.16 | 110 |
| 刚 | 0.15 | 111 |

Example assuming we have 100 symbols:

- 35 * 1 = 35 bits
- 17 * 3 = 51 bits
- 17 * 3 = 51 bits
- 16 * 3 = 48 bits
- 15 * 3 = 45 bits

Total: 230 bits
230 / 100 = 2.3 bits/symbol

# Efficiency Assessment of Huffman Coding

If we had a file with 350 我 characters , 170 爸 characters , 170 是 characters, 160 李 characters, and 150 刚 characters, how many total bits would we need to encode this file using 32 bit Unicode? Using our Huffman code?

You don’t need a calculator.

| Symbol | Frequency | Huffman Code |
|---|---|---|
| 我 | 0.35 | 0 |
| 爸 | 0.17 | 100 |
| 是 | 0.17 | 101 |
| 李 | 0.16 | 110 |
| 刚 | 0.15 | 111 |

2.30 bits per symbol for texts with this distribution

# Efficiency Assessment of Huffman Coding

If we had a file with 350 我 characters , 170 爸 characters , 170 是 characters, 160 李 characters, and 150 刚 characters, how many total bits would we need to encode this file using 32 bit Unicode? Using our Huffman code?

1000 total characters.

Space used:

- 32 bit Unicode: 32,000 bits.
- Huffman code: 2,300 bits.

Our code is 14 times as efficient!

- Can only encode strings with these 5 symbols.

| Symbol | Frequency | Huffman Code |
|---|---|---|
| 我 | 0.35 | 0 |
| 爸 | 0.17 | 100 |
| 是 | 0.17 | 101 |
| 李 | 0.16 | 110 |
| 刚 | 0.15 | 111 |

2.30 bits per symbol for texts with this distribution

# Huffman vs. Shannon-Fano

Shannon-Fano code below results in an average of 2.31 bits per symbol, whereas Huffman is only 2.3 bits per symbol.

- Huffman coded file is 0.35*1 + 0.65*3 = 2.3 bits per symbol.

| Symbol | Frequency | S-F Code | Huffman Code |
|---|---|---|---|
| 我 | 0.35 | 00 | 0 |
| 爸 | 0.17 | 01 | 100 |
| 是 | 0.17 | 10 | 101 |
| 李 | 0.16 | 110 | 110 |
| 刚 | 0.15 | 111 | 111 |

Strictly better than Shannon-Fano coding. There is NO downside to Huffman coding instead.

# Data Structures for Huffman Coding

Lecture 39, CS61B, Spring 2026

Today's Goal: Compression

Prefix Free Codes

Shannon Fano Codes (Extra)

**Huffman Coding**

- Core Idea
- **Data Structures for Huffman Coding**
- Huffman Coding in Practice

Compression Theory

- Compression is Hard
- Extreme Compression

LZW Style Compression (Extra)

# Prefix-Free Codes

Question: For encoding (bitstream to compressed bitstream), what is a natural data structure to use? Assume characters are of type Character, and bit sequences are of type BitSequence.

| | |
|---|---|
| space | 1 |
| E | 01 |
| T | 001 |
| A | 0001 |
| O | 00001 |
| I | 000001 |
| ... | |

I ATE: 0000011000100101

| | |
|---|---|
| space | 111 |
| E | 010 |
| T | 1101 |
| A | 1011 |
| O | 1001 |
| I | 1000 |
| ... | 0111 |

I ATE: 100011110111101010

# Prefix-Free Codes

Question: For encoding (bitstream to compressed bitstream), what is a natural data structure to use? chars are just integers, e.g. 'A' = 65. Two approaches:

- Array of BitSequence[], to retrieve, can use character as index.
- How is this different from a HashMap<Character, BitSequence>? Lookup in a hashmap consists of:
  - Compute hashCode.
  - Mod by number of buckets.
  - Look in a linked list.

Compared to HashMaps, Arrays are faster (just get the item from the array), but use more memory if some characters in the alphabet are unused.

I ATE: 0000011000100101

I ATE: 1000111101111101010

# Prefix-Free Codes

Question: For decoding (compressed bitstream back to bitstream), what is a natural data structure to use?

| space | 1 |
|---|---|
| E | 01 |
| T | 001 |
| A | 0001 |
| O | 00001 |
| I | 000001 |
| ... | |

I ATE: 0000011000100101

| space | 111 |
|---|---|
| E | 010 |
| T | 1101 |
| A | 1011 |
| O | 1001 |
| I | 1000 |
| ... | 0111 |

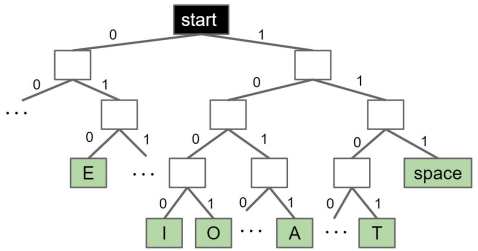
I ATE: 100011110111101010

# Prefix-Free Codes

Question: For decoding (compressed bitstream back to bitstream), what is a natural data structure to use?

- We need to look up **longest matching prefix**, an operation that Tries excel at.

| space | 1 |
|---|---|
| E | 01 |
| T | 001 |
| A | 0001 |
| O | 00001 |
| I | 000001 |
| ... | |




| space | 111 |
|---|---|
| E | 010 |
| T | 1101 |
| A | 1011 |
| O | 1001 |
| I | 1000 |
| ... | 0111 |

I ATE: 000001100010010 1

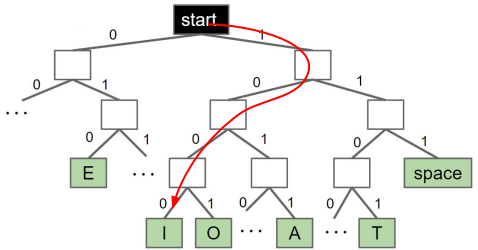
I ATE: 100011110111101010

# Prefix-Free Codes

Question: For decoding (compressed bitstream back to bitstream), what is a natural data structure to use?

- We need to look up **longest matching prefix**, an operation that Tries excel at.

| space | 1 |
|---|---|
| E | 01 |
| T | 001 |
| A | 0001 |
| O | 00001 |
| I | 000001 |
| ... | |




| space | 111 |
|---|---|
| E | 010 |
| T | 1101 |
| A | 1011 |
| O | 1001 |
| **I** | **1000** |
| ... | 0111 |

I ATE: 0000011000100101

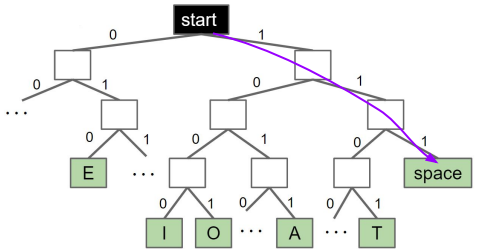
**I** ATE: ~~1000~~111101111101010

# Prefix-Free Codes

Question: For decoding (compressed bitstream back to bitstream), what is a natural data structure to use?

- We need to look up **longest matching prefix**, an operation that Tries excel at.

| space | 1 |
|---|---|
| E | 01 |
| T | 001 |
| A | 0001 |
| O | 00001 |
| I | 000001 |
| ... | |




| **space** | **111** |
|---|---|
| E | 010 |
| T | 1101 |
| A | 1011 |
| O | 1001 |
| I | 1000 |
| ... | 0111 |

I ATE: 000001100010010 1

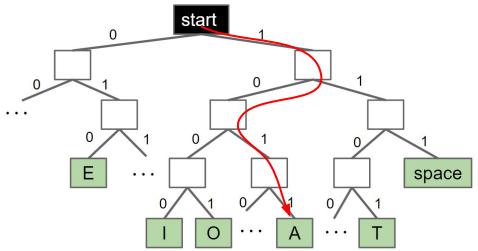
I ATE: ~~1000~~ ~~111~~ 10111101010

# Prefix-Free Codes

Question: For decoding (compressed bitstream back to bitstream), what is a natural data structure to use?

- We need to look up **longest matching prefix**, an operation that Tries excel at.

| space | 1 |
|---|---|
| E | 01 |
| T | 001 |
| A | 0001 |
| O | 00001 |
| I | 000001 |
| ... | |




| space | 111 |
|---|---|
| E | 010 |
| T | 1101 |
| **A** | **1011** |
| O | 1001 |
| I | 1000 |
| ... | 0111 |

I ATE: 000001100010010 1

I **A**TE: ~~1000~~ ~~1110~~ ~~11~~ 1101010

# Huffman Coding in Practice

Lecture 39, CS61B, Spring 2026

Today's Goal: Compression

Prefix Free Codes

Shannon Fano Codes (Extra)

**Huffman Coding**

- Core Idea
- Data Structures for Huffman Coding
- **Huffman Coding in Practice**

Compression Theory

- Compression is Hard
- Extreme Compression

LZW Style Compression (Extra)

# Huffman Compression

Two possible philosophies for using Huffman Compression:

1. For each input type (English text, Chinese text, images, Java source code, etc.), assemble huge numbers of sample inputs for that category. Use each corpus to create a standard code for English, Chinese, etc.
2. For every possible input file, create a unique code just for that file. Send the code along with the compressed file.

What are some advantages/disadvantages of each idea? Which is better?

```
$ java HuffmanEncodePh1 ENGLISH mobydick.txt
$ java HuffmanEncodePh1 BITMAP horses.bmp
```

```
$ java HuffmanEncodePh2 mobydick.txt
$ java HuffmanEncodePh2 horses.bmp
```

# Huffman Compression (Your Answers)

Two possible philosophies for using Huffman Compression:

1. Build one corpus per input type.
2. For every possible input file, create a unique code just for that file. Send the code along with the compressed file.

What are some advantages/disadvantages of each idea? Which is better?

- Ph1 If you standardize and then you have a file with uncommon letters, you'll have bad compression (may not have a good corpus to select that works well).
- Ph1 - maybe more space efficient? You don't have to send the tree along with your data.
- Ph2 - Only have subset of files in text file, maybe only 2 or 3, why would you need some big fancy tree?

# Huffman Compression (My Answers)

Two possible philosophies for using Huffman Compression:

1. Build one corpus per input type.
2. For every possible input file, create a unique code just for that file. Send the code along with the compressed file.

What are some advantages/disadvantages of each idea? Which is better?

- Approach 1 will result in suboptimal encoding.
- Approach 2 requires you to use extra space for the codeword table in the compressed bitstream.

For very large inputs, the cost of including the codeword table will become insignificant.

# Huffman Compression

Two possible philosophies for using Huffman Compression:

1. For each input type (English text, Chinese text, images, Java source code, etc.), assemble huge numbers of sample inputs for that category. Use each corpus to create a standard code for English, Chinese, etc.
2. For every possible input file, create a unique code just for that file. Send the code along with the compressed file.

In practice, Philosophy 2 is used in the real world.

# Huffman Compression Example [Demo Link]

Given **input text**: 我我刚刚刚是我是我刚李刚我李是爸李爸李是李我我李刚是我是刚爸是刚我爸我李是是李是我我刚爸是李我我我是爸我是我爸是我爸是我是刚我是爸刚爸我刚我我刚爸我我爸我刚爸爸李李李李我我爸李我我刚爸李我我李我爸我我

Step 1: Count frequencies.

Step 2: Build encoding array and decoding trie.

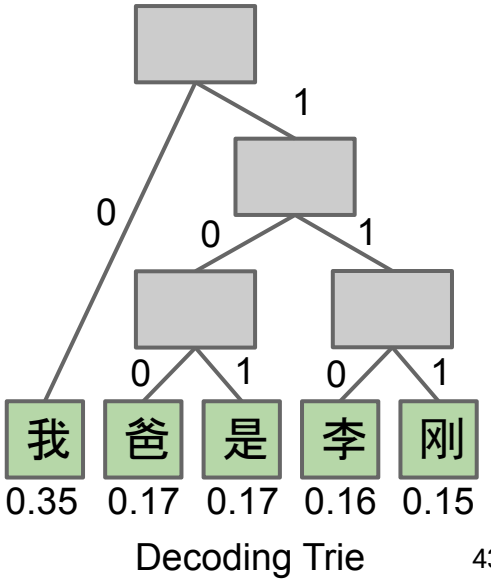
Step 3: Write decoding trie to output.huf.

Step 4: Write codeword for each symbol to output.huf.

→ See `writeTrie` in this code if you're curious.

**Output bits**: 010101010101001…00111111111101…

Decoding Trie | Codewords




Decoding Trie

# Huffman Decompression Example [Demo Link]

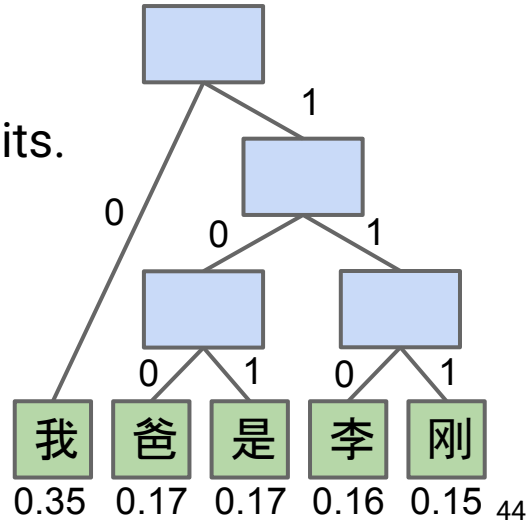
Given **input bitstream**: `010101010101001...00111111111101...`

Decoding Trie | Codewords

Step 1: Read in decoding trie.

Step 2: Use codeword bits to walk down the trie, outputting symbols every time you reach a leaf.

- Note: Symbols are really just bits!
  - 我 is `11100101100010010010001` in Unicode.
  - "Outputting 我" actually means outputting these 32 bits.




**Output symbols**: 我我刚刚刚是…

- Output bits: `11100101100010010010001`...

# Huffman Coding Summary

Given a file X.txt that we'd like to compress into X.huf:

- Consider each b-bit symbol (e.g. 8-bit chunks, Unicode characters, etc.) of X.txt, counting occurrences of each of the $2^b$ possibilities, where b is the size of each symbol in bits.
- Use Huffman code construction algorithm to create a decoding trie and encoding map. Store this trie at the beginning of X.huf.
- Use encoding map to write codeword for each symbol of input into X.huf.

To decompress X.huf:

- Read in the decoding trie.
- Repeatedly use the decoding trie's `longestPrefixOf` operation until all bits in X.huf have been converted back to their uncompressed form.





See Huffman.java for an example implementation on 8-bit symbols.

# Compression is Hard

Lecture 39, CS61B, Spring 2026

Today's Goal: Compression

Prefix Free Codes

Shannon Fano Codes (Extra)

Huffman Coding

- Core Idea
- Data Structures for Huffman Coding
- Huffman Coding in Practice

**Compression Theory**

- **Compression is Hard**
- Extreme Compression

LZW Style Compression (Extra)

# Compression Algorithms (General)

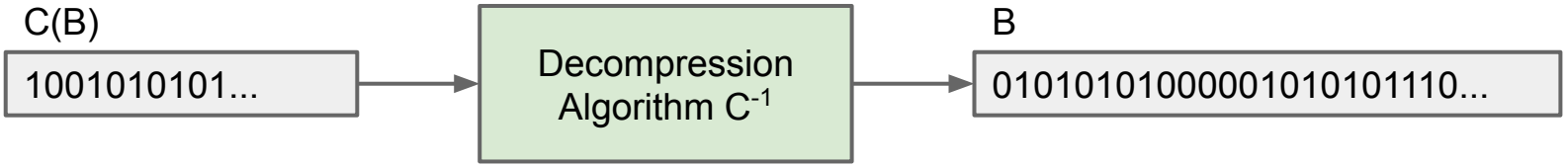
The big idea in Huffman Coding is representing common symbols with small numbers of bits.

Many other approaches, e.g.

- Run-length encoding: Replace each character by itself concatenated with the number of occurrences.
  - Rough idea: XXXXXXXXXYYYYXXXXX -> X10Y4X5
  - Can compress some strings very very well, e.g. 10 million Xs in a row is just “X1000000”.
- LZW: Search for common repeated patterns in the input. See extra slides.

General idea: Exploit redundancy and existing order inside the sequence.

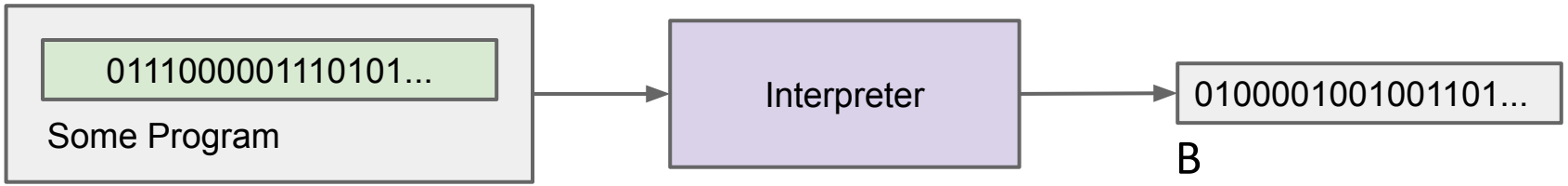
# Compression Model #1: Algorithms Operating on Bits




So far in lecture, we've modeled compression as shown above.

Another useful model for compression (model #2) is as shown below:

- Instead of feeding "compressed bits" into a "decompression algorithm", we'll feed "some program" into an "interpreter".



# Interesting Question #1

Given a bitstream, is there a limit to how much we can compress it?

# Comparing Compression Algorithms

Different compression algorithms achieve different compression ratios on different files.

We'd like to try to compare them in some nice way.

- To do this, we'll need to refine our model from slide 4 to be a bit more sophisticated.

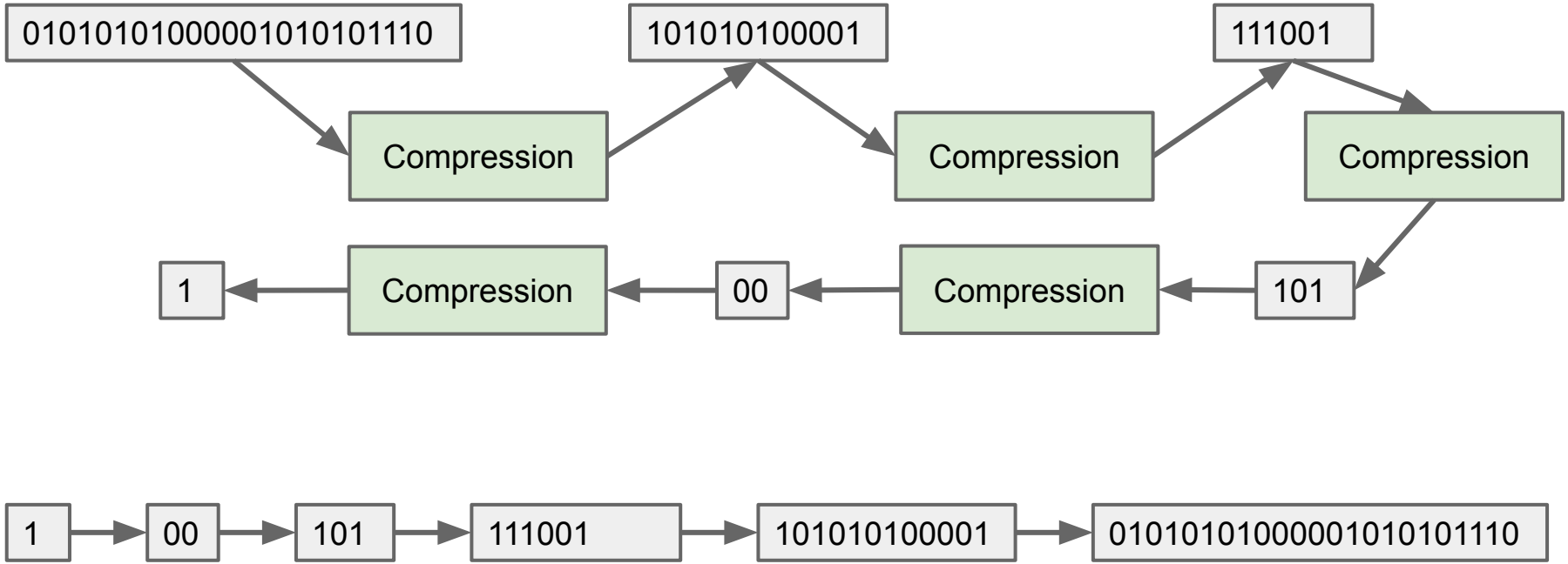
Let's start with a straightforward puzzle.

# SuperZip: yellkey.com/attorney

Suppose an algorithm designer says their algorithm SuperZip can compress any bitstream by 50%. Why is this impossible?

# Universal Compression: An Impossible Idea

Simplest argument: If true, they'd be able to compress any bitstream down to a single bit. Interpreter would have to be able to do the following (impossible) task for ANY output sequence.



# Interesting Question #2

Are some bitstreams more compressible than others?

For example, which of these are more compressible?

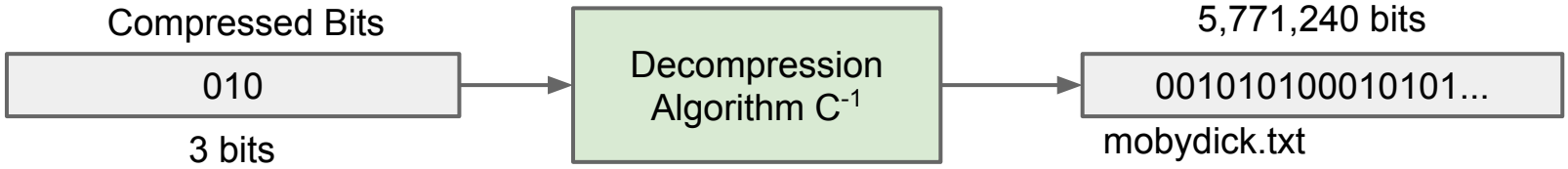
- 00000000000000000000000000000000000000000000000000000000000000000000…
- 00101010001010100101010001101000011001010010000001010000011100100111011110110101…
- 11001011100100101001111011001011100010110011111111000010011011101111110101110101…

# A Sneaky Situation / Thought Experiment

Imagine a compression algorithm that has a special case:

- If the compressed sequence is 010, simply give back the text of mobydick.txt

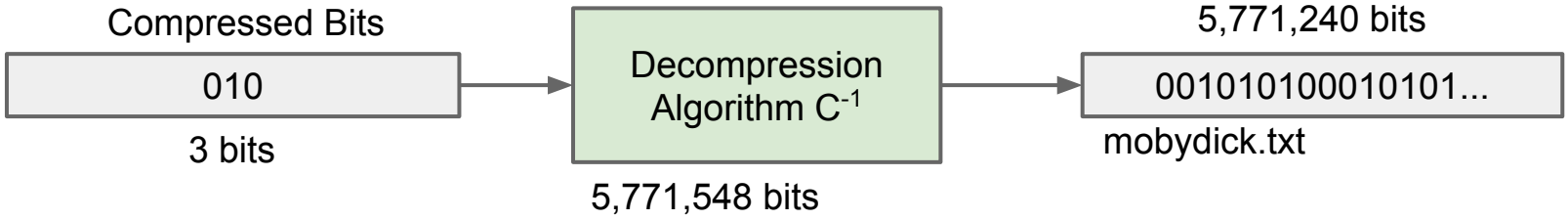
```
public static BitSet decompress(BitSet input, int inputLength) {
   String inputBits = bitSetToString(input, inputLength);

   if (inputBits.equals("010")) {
       return stringToBitSet("00101010001010100101010001101000110010100...");
   }
   ...
```



# A Sneaky Situation / Thought Experiment

How can we change our “model” for the compression process to avoid this trickery?

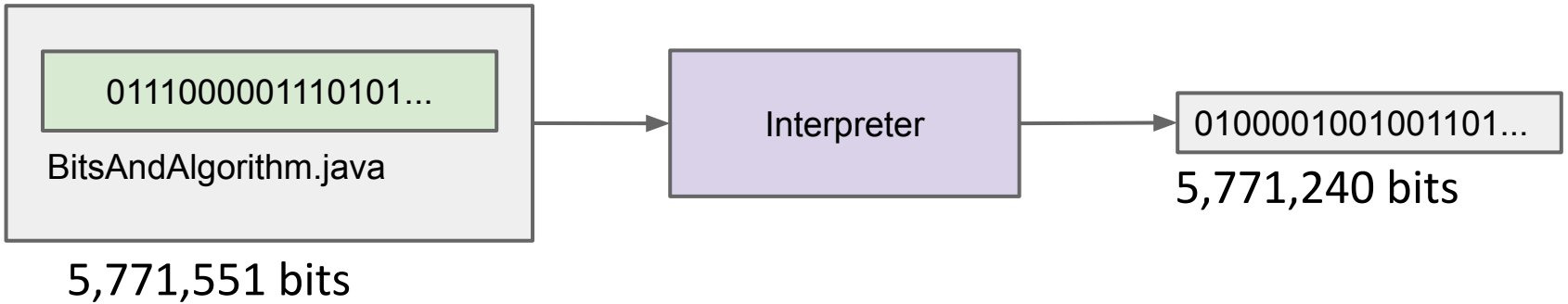
```
public static BitSet decompress(BitSet input, int inputLength) {
   String inputBits = bitSetToString(input, inputLength);

   if (inputBits.equals("010")) {
       return stringToBitSet("00101010001010100101010001101000011001010O");
   }
   ...
```



# Compression Model #2

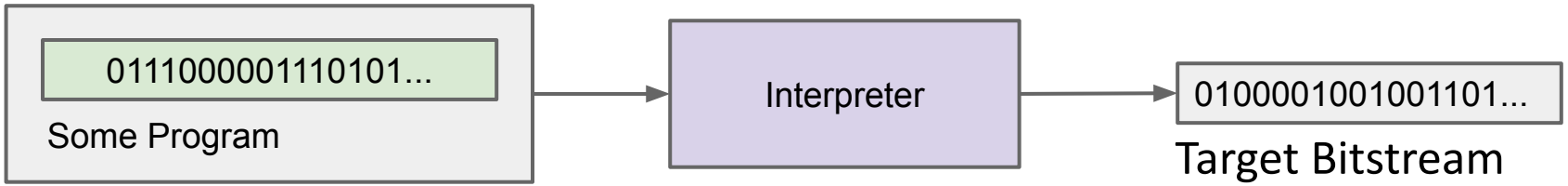
The fix: **Let's model the compressed data as including the source code for the decompression algorithm.**

The nice thing with this approach: We have a universal algorithm (the interpreter) that does the decompression process.



In this more general framing for compression, we can ask the interesting question:

- What is the shortest program that can output a given bitstream?

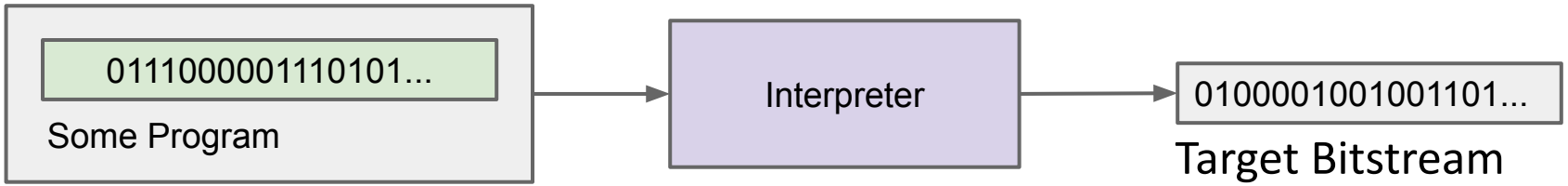


# Compression Theory

Suppose we want to find the shortest program that outputs a target bitstream X.

The theory behind compression is very deep. For example:

- There exists a shortest possible program that outputs X.
- The theoretical number of bits you need to output X is independent (to within an **additive** constant factor) of the programming language you use.
- Only a tiny fraction of target bitstreams can be generated by a program shorter than the original bitstream.
  - In other words: Compression is almost always impossible (on random data). Why? There are vastly more long sequences than short ones.



# Compression is Hard

Lecture 39, CS61B, Spring 2026

Today's Goal: Compression

Prefix Free Codes

Shannon Fano Codes (Extra)

Huffman Coding

- Core Idea
- Data Structures for Huffman Coding
- Huffman Coding in Practice

**Compression Theory**

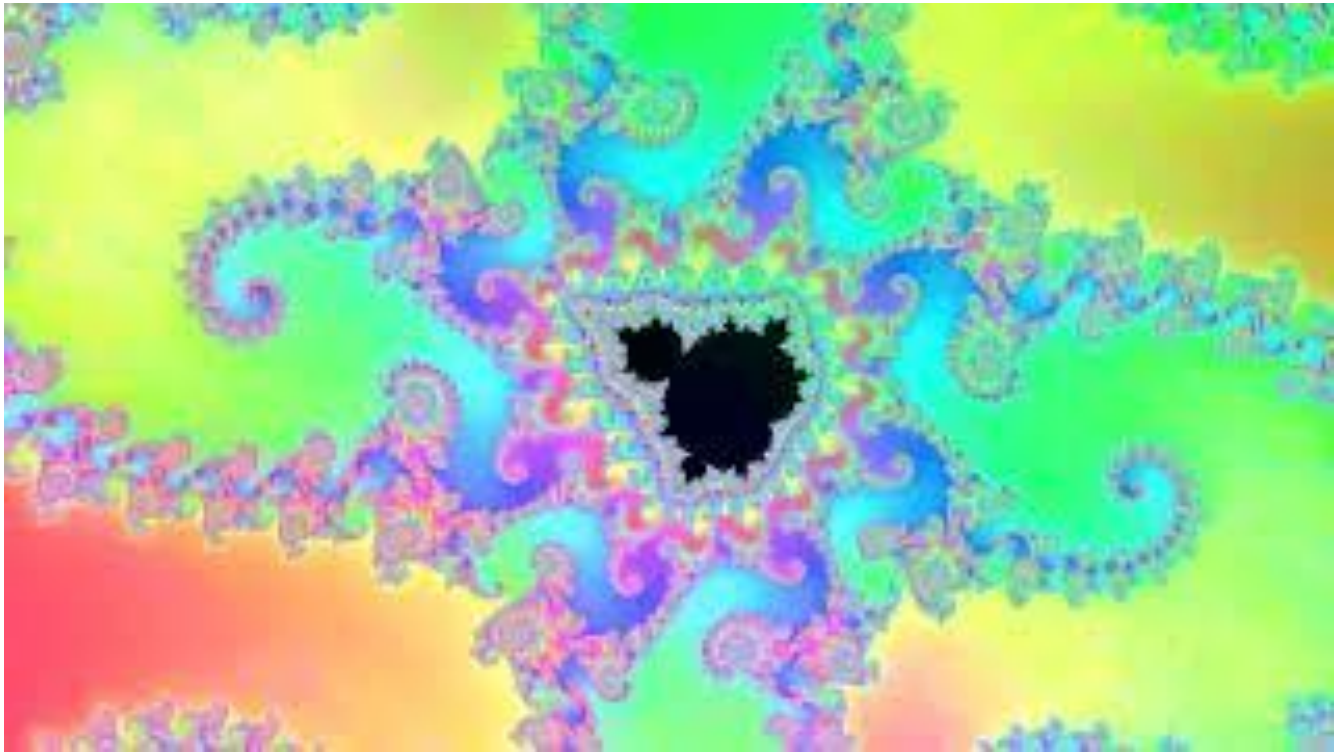
- Compression is Hard
- **Extreme Compression**

LZW Style Compression (Extra)

# At the Other Extreme

Yet some data is capable of extreme compression. Consider this video:

- https://youtu.be/1hEh9RR6Z30

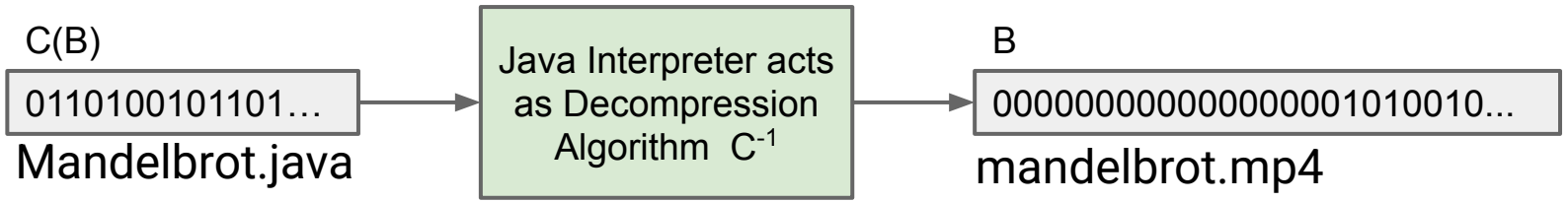


# Compression

The video file of that Zoomed fractal is 5,973,182,440 bits long.

- But it is possible to compress this huge bit stream to only 37,504 bits.

```
❯ ls -ltr
total 1458336
-rw-r--r--@ 1 hug  staff        286 Nov 24 07:56 lec38.iml
-rw-r--r--@ 1 hug  staff        997 Nov 24 07:59 pom.xml
drwxr-xr-x@ 4 hug  staff        128 Nov 24 07:59 target
-rw-r--r--@ 1 hug  staff       4688 Nov 24 08:04 Mandelbrot.java
-rw-r--r--@ 1 hug  staff  746647805 Nov 24 09:02 mandelbrot_4k.mp4
```



# Mandelbrot

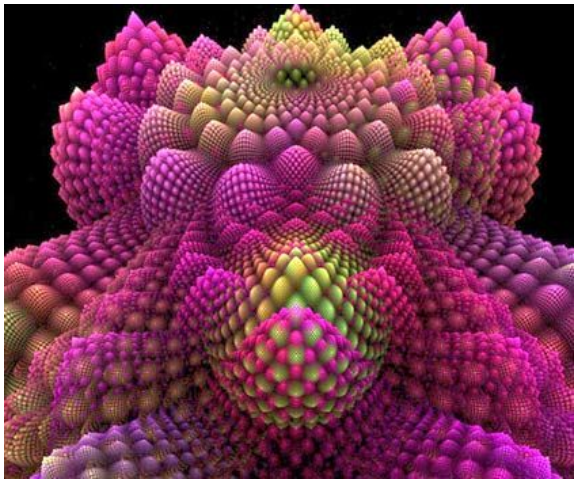
Here is the entire source for the code that generated the video we saw:

- https://github.com/Berkeley-CS61B/lectureCode-sp26/blob/main/lec39_compression/Mandelbrot.java

# Fractals

In the 1970s, Benoit Mandelbrot built demonstrations that very short programs could generate highly complex visual patterns.

- Biological processes exploit these same ideas. Short sequences of DNA give rise to interesting spatial (and other) patterns.



# Music from Simple Rules

This notion of complexity from simple rules is arguably more interesting (and alarming) when applied towards sound generation.

In this clip, we can hear some fractal sounds.

- This is excerpted from this video from Ville-Matias Heikkilä (viznut), in which we'll explore fractal sounds.

See following two slides if you want to play around with this yourself.

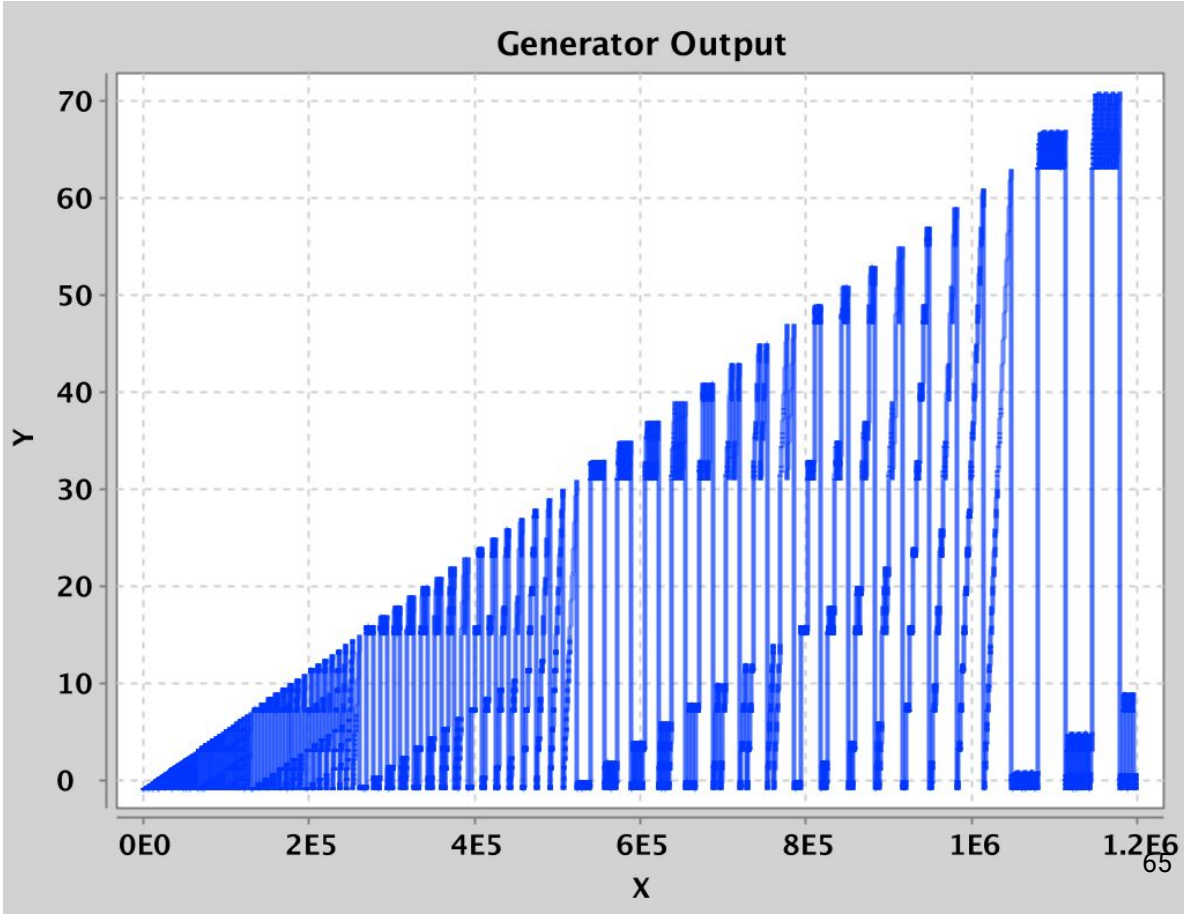
# Example Output

To experiment, you can use this basic sound playing tool: https://joshh.ug/61b/vsp/

- Next slide gives some examples you can try.

To the right is is a typical sequence generated by one of these simple programs.

- Your speaker is literally bulging in and out following this waveform.



# Music from Simple Rules

A couple of interesting programs:

- `return ((t >> 4 | t & (t >> 5) / (t >> 7 - (t >> 15) & -t >> 7 - (t >> 15))) % 256)` (link)

- ```
  return (
    ((t * ("36364689"[t >> 13 & 7] & 15)) / 12 & 128) +
    (((((t >> 12) ^ (t >> 12) - 2) % 11 * t) / 4 | t >> 13) & 127)
  ) % 256;
  ```
  (link)

- ```
  return (35*(3e3/(y = t & 16383) & 1)
      + (x = t * "6689"[t >> 16 & 3] / 24 & 127) * y / 4e4
      + ((t >> 8 ^ t >> 10 | t >> 14 | x) & 63)) % 1024;
  ```
  (link)

- (a bit too long for the slide) (link)

# Compression Theory

Suppose we want to find the shortest program that outputs a target bitstream X.

The theory behind compression is very deep. For example:

- There exists a shortest possible program that outputs X.
- The number of bits you need to output X is independent (to within a constant **additive** factor) of the programming language you use.
- There exists no algorithm `findShortestProgram(X)` that can find the shortest program for any X.
- There exists no algorithm `findShortestProgramLength(X)` that can find the length of the shortest program for any X.
- Suppose we had an algorithm for efficiently finding the longest path in a graph. Then we would also be able to write an efficient `findJavaCodeThatOutputs(X, numCycles, numBytes)` if such a Java program exists.
- Kooky (?) idea: Consciousness is a form of compression (Link).

# LZW Style Compression (Extra)

Lecture 39, CS61B, Spring 2026

# Thought Experiment

How might we compress the following bitstreams (underlines for emphasis only)?

- B=”aababcabcdabcdeabcdefabcdefgabcdefgh”?
- B=”abababababababababababababababab”?
- B=”aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa”?

# The LZW Approach

Key idea: Each ***codeword*** represents multiple symbols.

- Start with ‘trivial’ codeword table where each codeword corresponds to one ASCII symbol.
- Every time a codeword X is used, record a new codeword Y corresponding to X concatenated with the next symbol.

Demo Example: http://goo.gl/68Dncw

# LZW


Named for inventors Limpel, Ziv, Welch.

- Related algorithm used as a component in many compression to[illegible]
  .gif files, .zip files, and more.
- Once a hated algorithm because of attempts to enforce licensing fees. Patent expired in 2003.

Our version in lecture is simplified, for example:

- Assumed inputs were ≤ 0x7f (7 bit input) and also provided 8 bit outputs (real LZW can have variable length outputs).
- Didn’t say what happens when table is full (many variants exist).

# LZW

Neat fact: You don’t have to send the codeword table along with the compressed bitstream.

- Possible to reconstruct codeword table from C(B) alone.

LZW decompression example:

http://goo.gl/fdYU9C